

Python Dictionary Practice

Real-Life & API-Style Exercises

1 Basic Dictionary Practice

1. Create a dictionary to store a student's name, age, phone, and city.
2. Print each value using dictionary keys.
3. Access one existing key using `[]` and one using `.get()`.
4. Try accessing a key that does not exist using both methods.
5. Add a new key "email" to the dictionary.
6. Update the phone number.
7. Check if the key "address" exists.
8. If "salary" does not exist, print 0 using safe access.

2 User Input & Dynamic Dictionary

1. Write a program that asks the user to enter key-value pairs.
2. Stop the program when user enters q.
3. Prevent duplicate keys.
4. Print all entered data at the end.

3 Looping Through Dictionary

1. Print all keys from the dictionary.
2. Print all values from the dictionary.
3. Print key and value together in format:

```
1 name => Ram Thapa
2 age  => 35
```

4 Deleting Dictionary Data

1. Remove the key "age" using pop.
2. Store the removed value and print it.
3. Remove the last inserted item using popitem.

5 Nested Dictionary Practice

```
1 user_info = {
2     "name": "Ram Thapa",
3     "address": {
4         "permanent": "Kathmandu",
5         "temporary": {
6             "city": "Hetauda",
7             "ward": 12
8         }
9     }
10 }
```

1. Print the temporary city.
2. Print the ward number using only get().
3. Add a new key "postal_code" inside temporary address.

6 Product API Style Data

```
1 products = [
2     {"id": 1, "name": "Smart Watch", "price": 3500, "stock": 10,
3      "category": "electronics"},
4     {"id": 2, "name": "Laptop", "price": 75000, "stock": 3, "
5      category": "electronics"},
6     {"id": 3, "name": "Headphone", "price": 1500, "stock": 0, "
7      category": "electronics"},
8 ]
```

```

5     {"id": 4, "name": "Notebook", "price": 120, "stock": 50, "
      category": "stationery"},
6     {"id": 5, "name": "Pen", "price": 30, "stock": 100, "category
      ": "stationery"},
7     {"id": 6, "name": "Office Chair", "price": 12000, "stock": 5,
      "category": "furniture"}
8 ]

```

Questions

1. Print all product names.
2. Find the most expensive product.
3. Find the cheapest product.
4. Print products costing more than 5000.
5. Print products that are out of stock.
6. Print products with their category.
7. Count number of products per category.
8. Calculate total stock value (price $\times$ stock).
9. Increase price of electronics by 10%.
10. Add key "status":
 - stock = 0 $\rightarrow$ Out of Stock
 - stock $\leq$ 5 $\rightarrow$ Low Stock
 - else $\rightarrow$ Available
11. Remove discontinued products (stock = 0).

7 Nested Orders Data

```

1 orders = [
2     {
3         "order_id": 101,
4         "customer": {"name": "Ram", "city": "Kathmandu"},
5         "items": [

```

```

6         {"name": "Laptop", "price": 75000, "qty": 1},
7         {"name": "Mouse", "price": 800, "qty": 2}
8     ]
9 },
10 {
11     "order_id": 102,
12     "customer": {"name": "Sita", "city": "Pokhara"},
13     "items": [
14         {"name": "Notebook", "price": 120, "qty": 5}
15     ]
16 }
17 ]

```

1. Print customer names.
2. Print all ordered item names.
3. Calculate total amount for each order.
4. Find order with highest total amount.

8 Backend Thinking Challenges

1. Ask user for product name and display its details.
2. Ask user for category and filter products.
3. Convert product list into dictionary using product id as key.

9 Open-Ended Challenges

1. Create your own shop data with nested dictionary.
2. Build a simple billing system using dictionary.
3. Debug the following code:

```

1 for product in products:
2     if product["price"] > "5000":
3         print(product["name"])

```